

Natural Language Processing

6. Neural language modeling and dense embeddings

Paolo Torrioni

Fall 2025

Notice

Credits

The present slides are largely an adaptation of existing material, including slides by [Jurafsky & Martin](#), [Manning](#), [Potts & MacCartney](#), [Black & Mortensen](#), and others, to whom I am very grateful.

Downloading and sharing

A copy of these slides can be downloaded from [virtuale](#) and stored for personal use only. Please do not redistribute.

Table of Contents

- Neural Language Modeling
- Word2vec
- Evaluating Embeddings
- Properties of Embeddings
- Analysing Embeddings

Neural Language Modeling

From Sparse to Dense Representations

TF-IDF and PPMI vectors are

- **long** (length $|V| = 20,000$ to $50,000$)
- **sparse** (most elements are zero)

Alternative: dense vectors which are

- **short** (length = 50 to 1,000)
- **dense** (most elements are non-zero)

Why?

Sparse vs Dense Vectors

- Short vectors may be easier to use as **features** in machine learning (less weights to tune)
- Dense vectors may **generalize** better than storing explicit counts
 - Idea: store “most” of the important information in a few dimensions
- They may do better at capturing **synonymy**:
 - **car** and **automobile** are synonyms; but are distinct dimensions
 - a word with **car** as a neighbor and a word with **automobile** as a neighbor should be similar, but aren't
- **In practice, they work better**
- Some popular embeddings you can download
 - **Word2vec** (Mikolov et al., 2013)
 - **GloVe** (Pennington et al., 2014)
 - **fastText** (Mikolov et al., 2017)
- Build on ideas from *neural language modeling*

Neural Language Modeling

- Statistical language modeling suffers from **curse of dimensionality**
 - Word sequences in the test set are likely different from those seen during training
- A lot of information in training corpus not used!
- Seeing

The cat is walking in the bedroom

in the training corpus should help generalize to make

A dog was running in a room

almost as likely

A neural probabilistic language model. Y Bengio et al, JMLR 2003

Feedforward Network

- The idea is to **predict** rather than **count**
- To illustrate: **feedforward network**
 - a **hidden layer h** formed of **hidden units h_i**
 - each layer is **fully-connected**

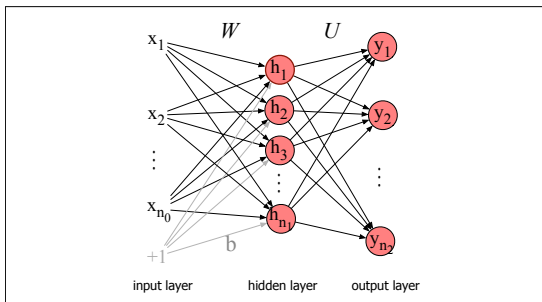


Figure 7.8 A simple 2-layer feedforward network, with one hidden layer, one output layer, and one input layer (the input layer is usually not counted when enumerating layers).

Feedforward Network

- n_0 inputs $\mathbf{x} = [x_1, \dots, x_{n_0}] \in \mathbb{R}^{n_0}$
- n_1 hidden neurons $\mathbf{h} = [h_1, \dots, h_{n_1}] \in \mathbb{R}^{n_1}$
 - $\mathbf{h}$ forms a *representation* of the input
- n_2 outputs $\mathbf{y} = [y_1, \dots, y_{n_2}] \in \mathbb{R}^{n_2}$
- parameters $\theta = \mathbf{W} \in \mathbb{R}^{n_1 \times n_0}$; $\mathbf{b} \in \mathbb{R}^{n_1}$; $\mathbf{U} \in \mathbb{R}^{n_2 \times n_1}$
- $\mathbf{h} = \tanh(\mathbf{W}\mathbf{x} + \mathbf{b})$
- $\mathbf{y} = \text{softmax}(\mathbf{U}\mathbf{h})$

Neural Language Modeling

To **predict** the next word given a sequence w_1^n :

- Associate each $w \in V$ with a *word feature vector* $e(w) \in \mathbb{R}^d$
 - **Distributed representation** of words
 - Stored in an **embedding matrix** $E \in \mathbb{R}^{d \times |V|}$
- Concatenate the vectors corresponding to the input sequence w_1^n
 - $x = [e(w_1); \dots; e(w_n)]$
 - $n_0 = n \times d$
- Use a feedforward network to produce a softmax distribution over V , pointing to the likeliest next word given w_1^n
 - $n_2 = |V|$
 - $y = P(w_{n+1} | w_1^n)$
- All the network parameters are learned **simultaneously**
- Why does it work?
 - **Similar words** expected to have **similar feature vectors**
 - The probability function is a *smooth* function of such features
 - A small change in the features will induce a small change in the probabilities

Neural Language Modeling

- The sequence w_1^n is a moving window that sees n words into the past
- Words can be represented using **one-hot encoding**
- The word with index i in V selects the i -th element (column) from E
- $e_i = E \times w_i$

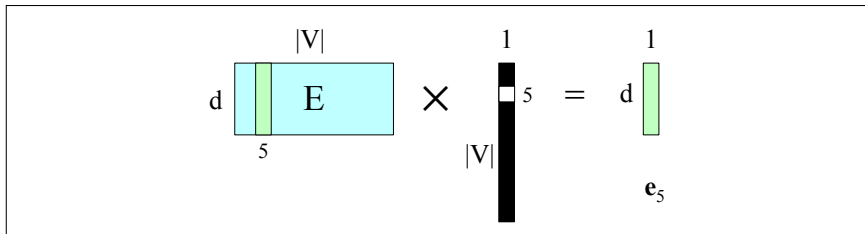


Figure 7.16 Selecting the embedding vector for word V_5 by multiplying the embedding matrix E with a one-hot vector with a 1 in index 5.

Neural Language Modeling

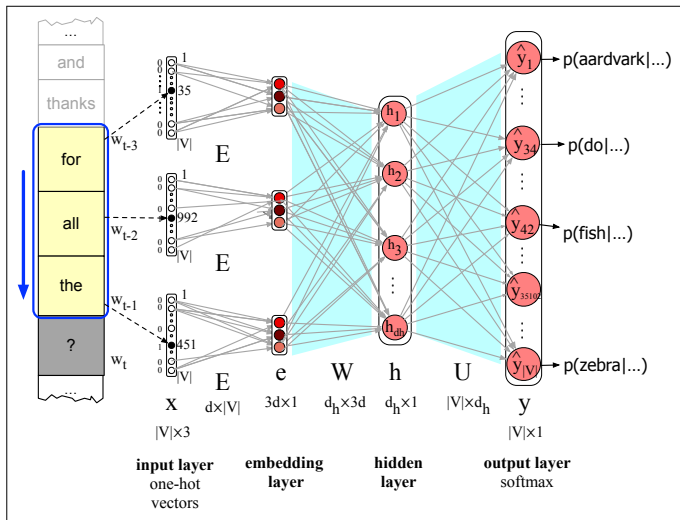


Figure 7.13 Forward inference in a feedforward neural language model.

Neural Language Modeling

Training:

- Initialize with small random weights with 0 mean and unit variance
- Take as input a very long text and concatenate all the sentences
- Predict one word at a time, iteratively
 - At each word w_t use **cross-entropy (negative log likelihood) loss**
 - Backpropagate through $\theta = \mathbf{E}, \mathbf{W}, \mathbf{U}, \mathbf{b}$

Importantly:

- $\mathbf{E}$ can later be used as features for other tasks
- **Learn representations** based on vast amounts of mostly unlabeled training data (**self-supervision**)

Neural Language Modeling

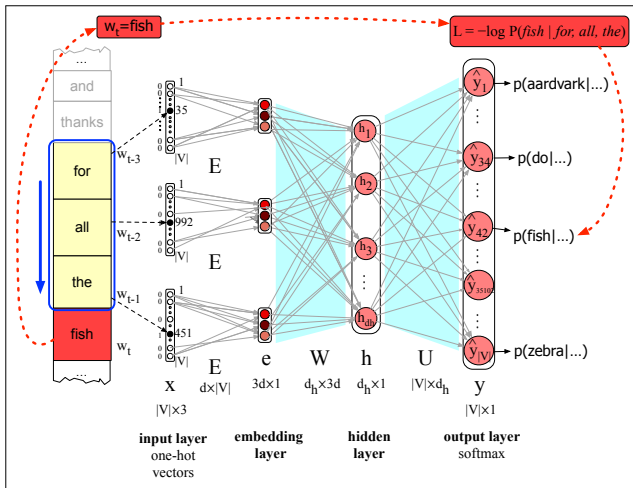


Figure 7.18 Learning all the way back to embeddings. Again, the embedding matrix E is shared among the 3 context words.

Word2vec

Word2vec

- Popular embedding method, simpler than neural language model
- Very fast to train
- **Code available**
- Learn a binary **prediction** task: *is a word w likely to show up near a target word c ?*
- Uses self-supervision
 - A word s near c acts as gold ‘correct answer’ to the question
- As usual, we don’t actually care about this task, but we’ll take the **learned classifier weights** as the **word embeddings**

Efficient Estimation of Word Representations in Vector Space, Mikolov et al., ICLR Workshops 2013

Distributed Representations of Words and Phrases and their Compositionality, Mikolov et al., NIPS 2013

Linguistic Regularities in Continuous Space Word Representations, Mikolov et al., NAACL-HLT 2013

Word2vec

word2vec is simpler than neural network language model

- **binary classification** task instead of word prediction
- **logistic regression** instead of MLP, easier training

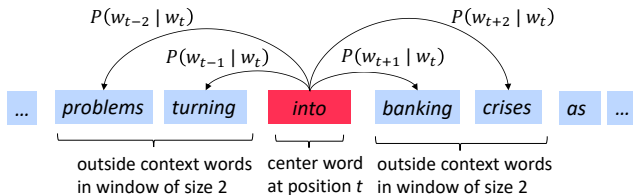
Intuition:

- 1 Treat target word and neighboring context as positive examples
- 2 Randomly sample other words to get negative examples
- 3 Use logistic regression to discriminate
- 4 Use learned weights as **embeddings**

Word2vec

More specifically:

- We have a large corpus of text
- Every word in a fixed vocabulary is represented by a vector
- Iterate through each word in the corpus
 - target: **center word, c**
 - context: **outside words, o**
- Use **similarity of the word vectors** for c and o to **calculate the probability** of o given c (or vice versa)
- Keep adjusting the word vectors to maximize this probability



Word2vec

Central idea: two vectors are similar if they have a high **dot product**

$$\text{similarity}(\mathbf{w}, \mathbf{c}) \approx \mathbf{c} \cdot \mathbf{w}$$

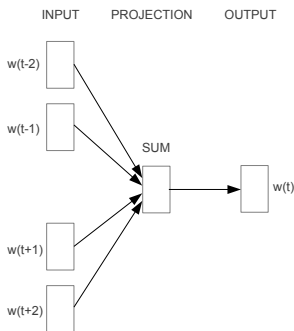
- Use sigmoid to turn dot product into probability
- Estimate probability that $\mathbf{c}$ is (not) a real context word for $\mathbf{w}$
 - for simplicity, assume all context words are independent

$$P(+|\mathbf{w}, \mathbf{c}) = \sigma(\mathbf{c} \cdot \mathbf{w}) = \frac{1}{1 + e^{-\mathbf{c} \cdot \mathbf{w}}} \quad (1)$$

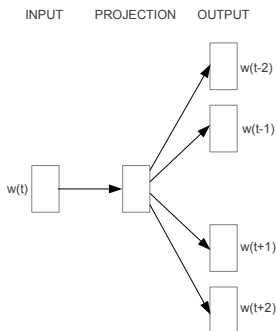
$$P(-|\mathbf{w}, \mathbf{c}) = 1 - P(+|\mathbf{w}, \mathbf{c}) = \sigma(-\mathbf{c} \cdot \mathbf{w}) = \frac{1}{1 + e^{\mathbf{c} \cdot \mathbf{w}}} \quad (2)$$

$$P(+|\mathbf{w}, \mathbf{c}_{1:L}) = \prod_{i=1}^L \sigma(\mathbf{c}_i \cdot \mathbf{w}) \Rightarrow \log P(+|\mathbf{w}, \mathbf{c}_{1:L}) = \sum_{i=1}^L \log \sigma(\mathbf{c}_i \cdot \mathbf{w}) \quad (3)$$

Model Variants



CBOW



Skip-gram

- **CBOW** (Continuous Bag Of Words)
 - predict center word from (bag of) outside words
- **Skip-grams**
 - predict outside words (position independent) given center word

Word2vec

- Assume all context words are independent
- For each position $t = 1, \dots, T$, predict context words within a window of fixed size m , given center word w_t (remember eq. 3)

$$\text{Likelihood} = L(\theta) = \prod_{t=1}^T \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} P(w_{t+j} | w_t; \theta)$$

- Objective function $J(\theta)$ is the (average) negative log likelihood:

$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} | w_t; \theta)$$

Word2vec

- To calculate $P(w_{t+j}|w_t; \theta)$, we use the **dot product**
- Use **two vectors** per word w :
 - u_w when w is an outside word (u_o)
 - v_w when w is a center word (v_c)
- Reason: easier optimization
- Average or concatenate both at the end (or keep just one)

$$P(o|c) = \frac{\exp(u_o^\top v_c)}{\sum_{w \in V} \exp(u_w^\top v_c)}$$

Word2vec Embeddings

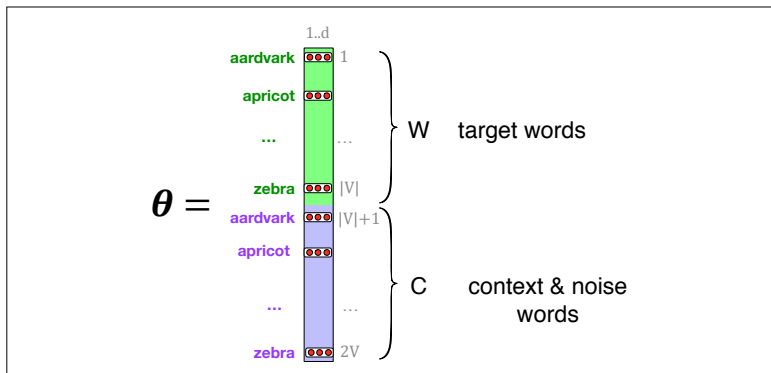
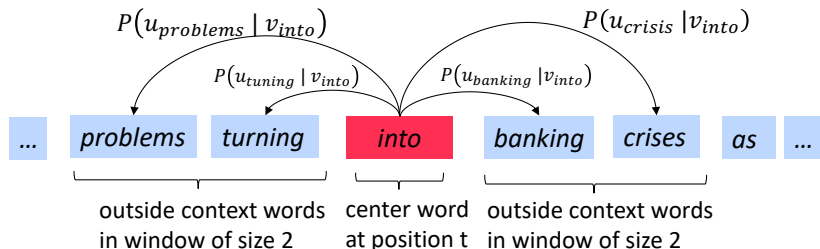


Figure 6.13 The embeddings learned by the skipgram model. The algorithm stores two embeddings for each word, the target embedding (sometimes called the input embedding) and the context embedding (sometimes called the output embedding). The parameter θ that the algorithm learns is thus a matrix of $2|V|$ vectors, each of dimension d , formed by concatenating two matrices, the target embeddings W and the context+noise embeddings C .

Word2vec

- Example windows and process for computing $P(w_{t+j}|w_t)$
- $P(u_{problems}|v_{into})$ short for $P(problems|into; u_{problems}, v_{into}, \theta)$



Negative Sampling

- Softmax too expensive because of normalization
- **Negative sampling**: for each positive example, create K negative samples using unigram probabilities (e.g., $K = 15$)
 - $P_{\alpha}(w) = \frac{Count(w)^{\alpha}}{\sum_w Count(w)^{\alpha}}$
 - $\alpha = 3/4$ gives rare noise words slightly higher probability
- Train **binary logistic regressions** for
 - a **true pair** (center word and word in its context window)
 - versus K **noise pairs** (the center word paired with a random word)

$$J_{SGNS}(o, v_c, U) = -\log(\sigma(u_o^{\top} v_c)) - \sum_{k=1}^K \log(\sigma(-u_k^{\top} v_c))$$

Skip-gram Algorithm

- ① Treat the target word and an outside word as positive examples
- ② Randomly sample other words in the lexicon to get negative samples
- ③ Use LR to train a classifier to distinguish those two cases
 - Start with V random 300-dimensional vectors as initial embeddings
 - Adjust weights to
 - **maximize** probability that real outside word appears
i.e., similarity of (t,c) pair drawn from positive data
 - **minimize** probability that random words appear around center
i.e., similarity of (t,c) pair drawn from negative data
- ④ Throw away classifier code and **use the weights as embeddings**

Skip-gram Algorithm

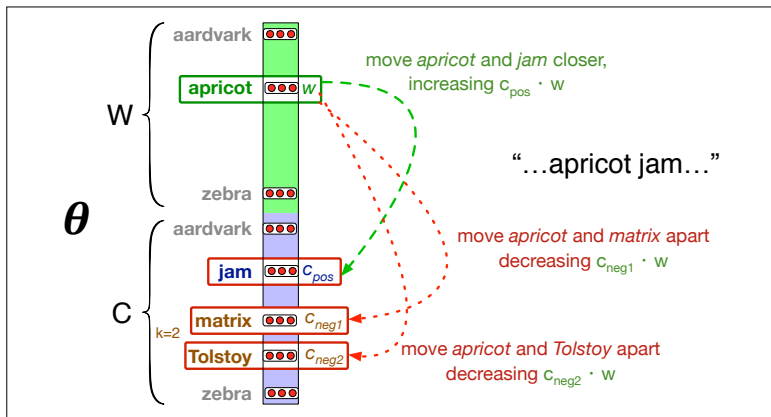


Figure 6.14 Intuition of one step of gradient descent. The skip-gram model tries to shift embeddings so the target embeddings (here for *apricot*) are closer to (have a higher dot product with) context embeddings for nearby words (here *jam*) and further from (lower dot product with) context embeddings for noise words that don't occur nearby (here *Tolstoy* and *matrix*).

Evaluating Embeddings

- Compare to human scores on word similarity-type tasks
- Some available datasets:
 - SimLex-999
 - Stanford Contextual Word Similarity (SCWS)
 - WordSim-353
 - TOEFL dataset
 - *Levied is closest in meaning to:
imposed, believed, requested, correlated*
- Analogy tasks
- Extrinsic evaluation

Properties of Embeddings

Properties of Embeddings

Similarity depends on window size C

- $C = \pm 2 \Rightarrow$ The nearest words to **Hogwarts**:
 - Sunnydale
 - Evernight
- $C = \pm 5 \Rightarrow$ The nearest words to **Hogwarts**:
 - Dumbledore
 - Malfoy
 - halfblood

Dependency-Based Word Embeddings, O Levy, Y Goldberg, ACL 2014

Properties of Embeddings

Two kinds of association between words

- **first-order co-occurrence** (syntagmatic association)
 - words that typically are nearby each other (**wrote**/**poem**)
- **second-order co-occurrence** (paradigmatic association)
 - words that have similar neighbours (**wrote**/**said**)

Embeddings Capture Relational Meaning!

Table 8: *Examples of the word pair relationships, using the best word vectors from Table 4 (Skip-gram model trained on 783M words with 300 dimensionality).*

Relationship	Example 1	Example 2	Example 3
France - Paris	Italy: Rome	Japan: Tokyo	Florida: Tallahassee
big - bigger	small: larger	cold: colder	quick: quicker
Miami - Florida	Baltimore: Maryland	Dallas: Texas	Kona: Hawaii
Einstein - scientist	Messi: midfielder	Mozart: violinist	Picasso: painter
Sarkozy - France	Berlusconi: Italy	Merkel: Germany	Koizumi: Japan
copper - Cu	zinc: Zn	gold: Au	uranium: plutonium
Berlusconi - Silvio	Sarkozy: Nicolas	Putin: Medvedev	Obama: Barack
Microsoft - Windows	Google: Android	IBM: Linux	Apple: iPhone
Microsoft - Ballmer	Google: Yahoo	IBM: McNealy	Apple: Jobs
Japan - sushi	Germany: bratwurst	France: tapas	USA: pizza

Efficient Estimation of Word Representations in Vector Space, Mikolov et al., ICLR Workshops 2013

Embeddings Capture Relational Meaning!

- $\text{vector}(\textit{king}) - \text{vector}(\textit{man}) + \text{vector}(\textit{woman}) \approx \text{vector}(\textit{queen})$
- $\text{vector}(\textit{Paris}) - \text{vector}(\textit{France}) + \text{vector}(\textit{Italy}) \approx \text{vector}(\textit{Rome})$

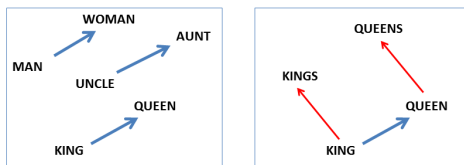


Figure 2: Left panel shows vector offsets for three word pairs illustrating the gender relation. Right panel shows a different projection, and the singular/plural relation for two words. In high-dimensional space, multiple relations can be embedded for a single word.

Analogy: caveats

For a $\mathbf{a} : \mathbf{b} :: \mathbf{a}^* : \mathbf{b}^*$ problem,

$$\mathbf{b}^* = \underset{x}{\operatorname{argmin}} \operatorname{distance}(\mathbf{x}, \mathbf{b} - \mathbf{a} + \mathbf{a}^*)$$

Analogy seems important. However:

- Closest value in word2vec or GloVe is usually not $\mathbf{b}^*$ but one of the 3 input words or their morphological variants
- Embedding spaces perform well with small distances, frequent words, certain relations. However, not always so well

“Palallelogram” method too simple to model human cognitive process of forming analogies

fastText

- An extension of word2vec
- **Subword models** to deal with OOV words and sparsity in languages with rich morphology
- Each word represented as itself + bag of constituent n-grams
 - For instance, with $n = 3$, **where** represented by <where>, <wh, whe, her, ere, re>
 - A skipgram embedding learned for each constituent n-gram
 - **where** represented by sum of all such embeddings
- Pre-trained embeddings for 157 languages are **available**

Enriching Word Vectors with Subword Information, Bojanowski, Grave, Joulin & Mikolov, TACL 2017

GloVe

Another widely used static embedding model

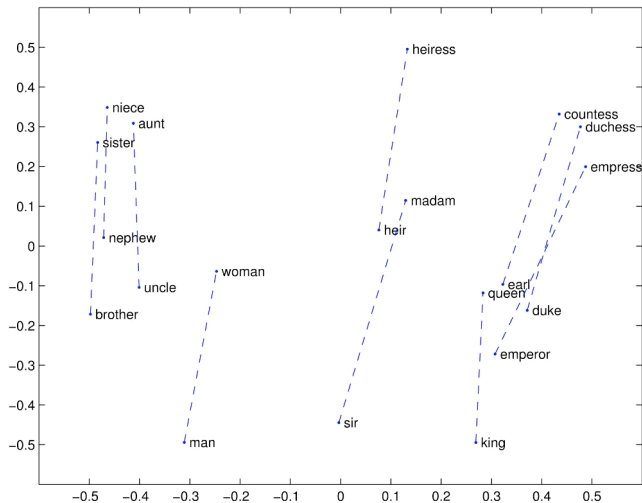
- Builds on global co-occurrence counts to retain corpus-wide statistical information
- Uses ratios of co-occurrence probabilities to encode meaning components

Table 1: Co-occurrence probabilities for target words *ice* and *steam* with selected context words from a 6 billion token corpus. Only in the ratio does noise from non-discriminative words like *water* and *fashion* cancel out, so that large values (much greater than 1) correlate well with properties specific to ice, and small values (much less than 1) correlate well with properties specific of steam.

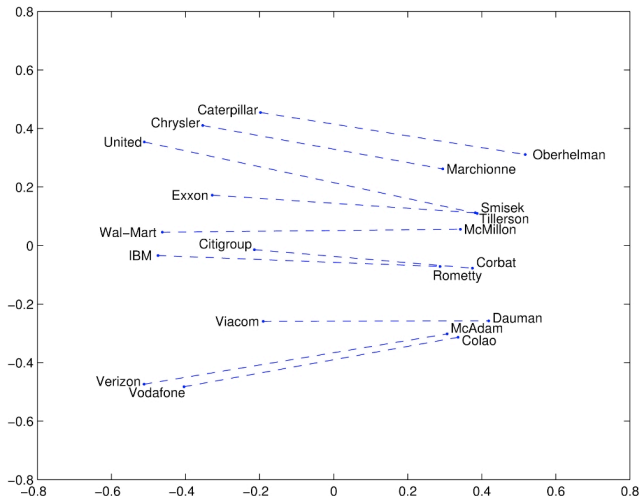
Probability and Ratio	$k = \text{solid}$	$k = \text{gas}$	$k = \text{water}$	$k = \text{fashion}$
$P(k \text{ice})$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \text{steam})$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k \text{ice})/P(k \text{steam})$	8.9	8.5×10^{-2}	1.36	0.96

GloVe: Global Vectors for Word Representation, Pennington, Socher & Manning, EMNLP 2014

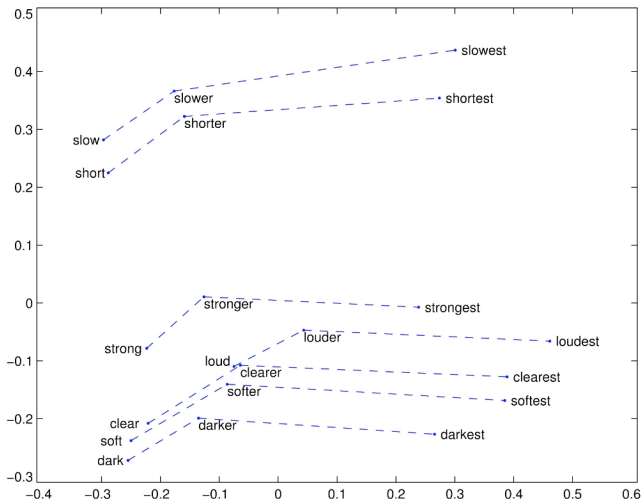
GloVe Visualizations



GloVe Visualizations: Company - CEO



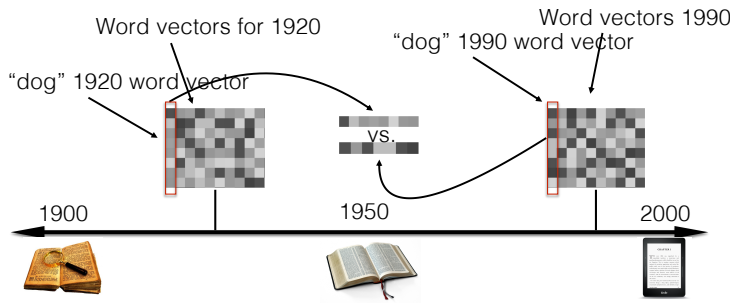
GloVe Visualizations: Superlatives



Analysing Embeddings

Embeddings Can Help Study Word History!

Train embeddings on old books to study changes in word meaning



Embeddings Can Help Study Word History!

Project 300 dimensions down into 2

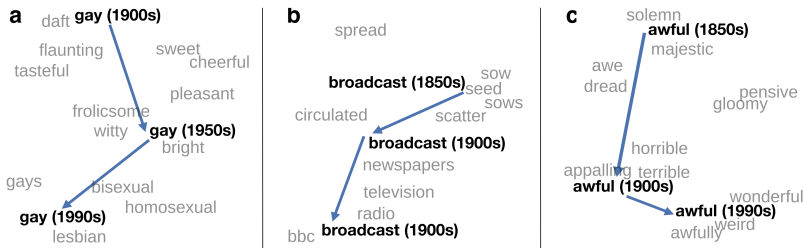


Figure 1: Two-dimensional visualization of semantic change in English using SGNS vectors.² **a**, The word *gay* shifted from meaning “cheerful” or “frolicsome” to referring to homosexuality. **b**, In the early 20th century *broadcast* referred to “casting out seeds”; with the rise of television and radio its meaning shifted to “transmitting signals”. **c**, *Awful* underwent a process of pejoration, as it shifted from meaning “full of awe” to meaning “terrible or appalling” (Simpson et al., 1989).

Diachronic Word Embeddings Reveal Statistical Laws of Semantic Change, WL Hamilton et al., ACL 2016

The Evolution of Sentiment Words

Negative words change faster than positive words

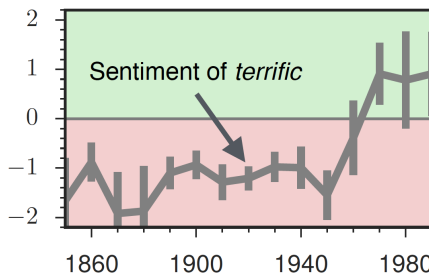


Figure 2: The sentiment of *terrific* changed from negative to positive over the last 150 years. Sentiment values and bootstrapped confidences were computed using SENTPROP on historical data (see Section 6).

Inducing Domain-Specific Sentiment Lexicons from Unlabeled Corpora, WL Hamilton et al., EMNLP 2016

Embeddings Reflect Cultural Bias

Embeddings also pinpoint sexism implicit in text

- $\text{vector}(\textit{father}) : \text{vector}(\textit{doctor}) :: \text{vector}(\textit{mother}) : \text{vector}(x)$

Extreme <i>she</i>	Extreme <i>he</i>		
1. homemaker	1. maestro	sewing-carpentry nurse-surgeon blond-burly giggle-chuckle sassy-snappy volleyball-football	Gender stereotype <i>she-he</i> analogies
2. nurse	2. skipper		registered nurse-physician
3. receptionist	3. protege		interior designer-architect
4. librarian	4. philosopher		feminism-conservatism
5. socialite	5. captain		vocalist-guitarist
6. hairdresser	6. architect	queen-king waitress-waiter	diva-superstar
7. nanny	7. financier		cupcakes-pizzas
8. bookkeeper	8. warrior		Gender appropriate <i>she-he</i> analogies
9. stylist	9. broadcaster		sister-brother
10. housekeeper	10. magician		mother-father
			ovarian cancer-prostate cancer convent-monastery

Figure 1: **Left** The most extreme occupations as projected on to the *she—he* gender direction on w2vNEWS. Occupations such as *businesswoman*, where gender is suggested by the orthography, were excluded. **Right** Automatically generated analogies for the pair *she-he* using the procedure described in text. Each automatically generated analogy is evaluated by 10 crowd-workers to whether or not it reflects gender stereotype.

Man is to computer programmer as woman is to homemaker? Debiasing word embeddings, T Bolukbasi et al., NIPS 2016

Embeddings Reflect Cultural Bias

- Implicit Association Test (Greenwald et al 1998): How associated are
 - **concepts** (*flowers* vs. *insects*) and **attributes** (*pleasantness* vs. *unpleasantness*)?
 - Studied by measuring timing latencies for categorization
- Psychological findings on US participants:
 - African-American names are associated with unpleasant words (more than European-American names)
 - Male names associated more with math, female names with arts
 - Old people's names with unpleasant, young people with pleasant words.
- Çalışkan et al. replication with embeddings:
 - African-American names (*Leroy*, *Shaniqua*) had a higher GloVe cosine with unpleasant words (*abuse*, *stink*, *ugly*)
 - European American names (*Brad*, *Greg*, *Courtney*) had a higher cosine with pleasant words (*love*, *peace*, *miracle*)
- **Embeddings reflect and replicate** all sorts of pernicious **biases**

Embeddings Reflect Cultural Bias

Table 1. Summary of Word-Embedding Association Tests. We replicated eight well-known IAT findings using word embeddings (rows 1 to 3 and 6 to 10); we also help explain prejudiced human behavior concerning hiring in the same way (rows 4 and 5). Each result compares two sets of words from target concepts about which we are attempting to learn with two sets of attribute words. In each case, the first target is found compatible with the first attribute, and the second target with the second attribute. Throughout, we use word lists from the studies we seek to replicate. N , number of subjects; N_T , number of target words; N_A , number of attribute words. We report the effect sizes (d) and

P values (P , rounded up) to emphasize that the statistical and substantive significance of both sets of results is uniformly high; we do not imply that our numbers are directly comparable with those of human studies. For the online IATs (rows 6, 7, and 10), P values were not reported but are known to be below the significance threshold of 10^{-2} . Rows 1 to 8 are discussed in the text; for completeness, this table also includes the two other IATs for which we were able to find suitable word lists (rows 9 and 10). We found similar results with word2vec, another algorithm for creating word embeddings, trained on a different corpus, Google News (see the supplementary materials).

Target words	Attribute words	Original finding				Our finding			
		Ref.	N	d	P	N_T	N_A	d	P
Flowers vs. insects	Pleasant vs. unpleasant	(5)	32	1.35	10^{-8}	25×2	25×2	1.50	10^{-7}
Instruments vs. weapons	Pleasant vs. unpleasant	(5)	32	1.66	10^{-10}	25×2	25×2	1.53	10^{-7}
European-American vs. African-American names	Pleasant vs. unpleasant	(5)	26	1.17	10^{-5}	32×2	25×2	1.41	10^{-8}
European-American vs. African-American names	Pleasant vs. unpleasant from (5)	(7)	Not applicable			16×2	25×2	1.50	10^{-4}
European-American vs. African-American names	Pleasant vs. unpleasant from (9)	(7)	Not applicable			16×2	8×2	1.28	10^{-3}
Male vs. female names	Career vs. family	(9)	39k	0.72	$<10^{-2}$	8×2	8×2	1.81	10^{-3}
Math vs. arts	Male vs. female terms	(9)	28k	0.82	$<10^{-2}$	8×2	8×2	1.06	.018
Science vs. arts	Male vs. female terms	(10)	91	1.47	10^{-24}	8×2	8×2	1.24	10^{-2}
Mental vs. physical disease	Temporary vs. permanent	(23)	135	1.01	10^{-3}	6×2	7×2	1.38	10^{-2}
Young vs. old people's names	Pleasant vs. unpleasant	(9)	43k	1.42	$<10^{-2}$	8×2	8×2	1.21	10^{-2}

Semantics derived automatically from language corpora contain human-like biases, A Caliskan et al., Science 2017

Embeddings as a Window Onto History

- Use the **Hamilton historical embeddings**
- The cosine similarity of embeddings for decade X for occupations (like teacher) to male vs female names is correlated with the actual percentage of women teachers in decade X
- Embeddings for competence adjectives are biased toward men
 - *Smart, wise, brilliant, intelligent, resourceful, thoughtful, logical, etc.*
 - This bias is slowly decreasing

Embeddings as a Window Onto History

Table 2. Top adjectives associated with women in 1910, 1950, and 1990 by relative norm difference in the COHA embedding

1910	1950	1990
Charming	Delicate	Maternal
Placid	Sweet	Morbid
Delicate	Charming	Artificial
Passionate	Transparent	Physical
Sweet	Placid	Caring
Dreamy	Childish	Emotional
Indulgent	Soft	Protective
Playful	Colorless	Attractive
Mellow	Tasteless	Soft
Sentimental	Agreeable	Tidy

Word embeddings quantify 100 years of gender and ethnic stereotypes, N Garg et al., PNAS 2018

Embeddings as a Window Onto History



Fig. 1. Women's occupation relative percentage vs. embedding bias in Google News vectors. More positive indicates more associated with women on both axes. $P < 10^{-10}$, $r^2 = 0.499$. The shaded region is the 95% bootstrapped confidence interval of the regression line. In this single embedding, then, the association in the embedding effectively captures the percentage of women in an occupation.

Embeddings as a Window Onto History

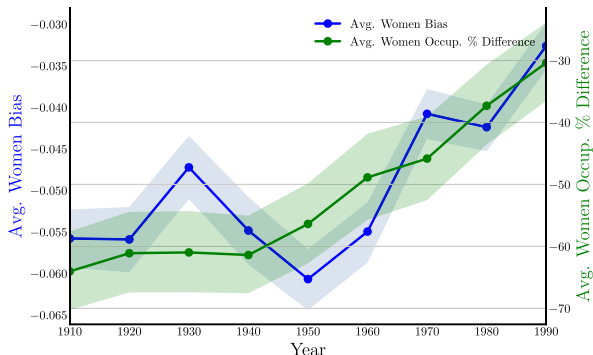


Fig. 2. Average gender bias score over time in COHA embeddings in occupations vs. the average percentage of difference. More positive means a stronger association with women. In blue is relative bias toward women in the embeddings, and in green is the average percentage of difference of women in the same occupations. Each shaded region is the bootstrap SE interval.

Embeddings as a Window Onto History

- Embeddings reflect ethnic stereotypes over time
- Princeton trilogy experiments
- Attitudes toward ethnic groups (1933, 1951, 1969) scores for adjectives
 - *industrious, superstitious, nationalistic*, etc.
- Cosine of Chinese name embeddings with those adjective embeddings correlates with human ratings

Embeddings as a Window Onto History

- Changes in framing: adjectives associated with Chinese

Table 3. Top Asian (vs. White) adjectives in 1910, 1950, and 1990 by relative norm difference in the COHA embedding

1910	1950	1990
Irresponsible	Disorganized	Inhibited
Envious	Outrageous	Passive
Barbaric	Pompous	Dissolute
Aggressive	Unstable	Haughty
Transparent	Effeminate	Complacent
Monstrous	Unprincipled	Forceful
Hateful	Venomous	Fixed
Cruel	Disobedient	Active
Greedy	Predatory	Sensitive
Bizarre	Boisterous	Hearty

Embeddings as a Window Onto History

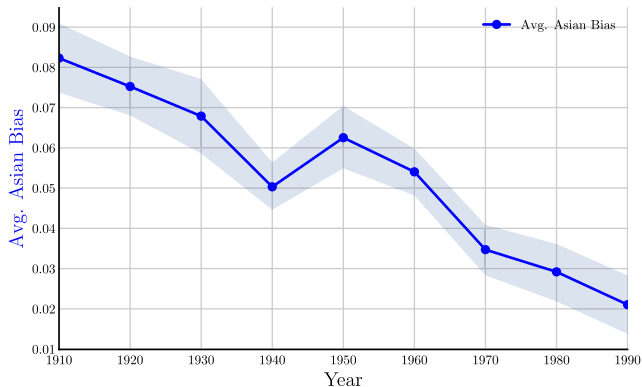


Fig. 6. Asian bias score over time for words related to outsiders in COHA data. The shaded region is the bootstrap SE interval.

Summary of Vector Space Models

Concepts or word senses

- Have a complex many-to-many association with words
- Have relations with each other
 - Synonymy, Antonymy, Superordinate
- But are hard to define formally (necessary & sufficient conditions)

Embeddings = vector models of meaning

- Can use **sparse** models (tf-idf, PPMI) or **dense** models (word2vec, GloVe, fastText)
- More fine-grained than just a string or index
- Especially good at modeling **similarity/analogy**
- They also encode cultural stereotypes (**representational harm**)

Questions?